Midterm Exam (40%)

- อนุญาตให้นำกระดาษ A4 หนึ่งแผ่นเข้าห้องสอบ (นักศึกษาต้องเขียนชื่อและรหัสนักศึกษาลงใน A4 พร้อมส่งคืนกระดาษ A4 ก่อนออกจากห้องสอบ)
- เตรียมอุปกรณ์เครื่องเขียนสำหรับการทำ Part 2 ให้พร้อม ไม่มีอุปกรณ์ให้ยืมในห้องสอบ

- Part 1 (15 points)
  - Software Engineering Foundation & Project Management
  - Software Process
  - Requirements Engineering
  - Interface Design
- Part 2 (25 points)
  - Requirements Analysis with Use Case Modeling and Data Modeling
    - Use case diagram
    - Use case narrative
  - Context DFD



ITDS261 Introduction to Software Engineering

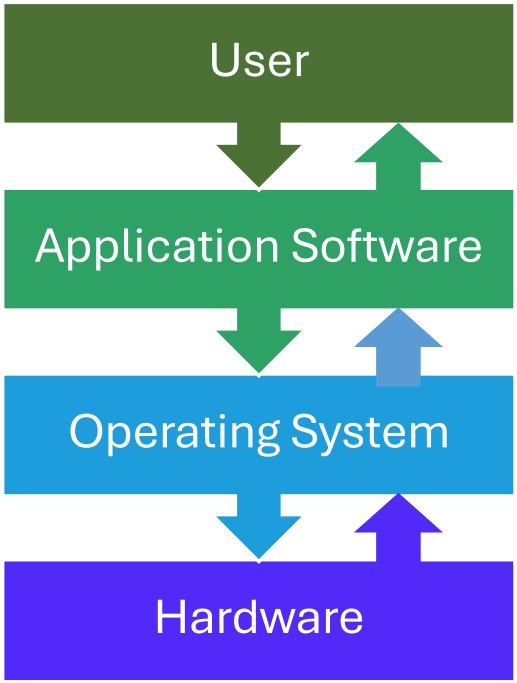
Lecture 1 – Software Engineering Foundation

Photo: https://www.flickr.com/photos/ninara/38707088530

Software and Its Components

Definitions:

1) A collection of *instructions* that provide features, function and performance
2) Data structures that enable the programs to manipulate information
3) A descriptive information *(document)* that describes the operation and use of the programs



Reference: Pressman, R.S. (2015) *Software engineering: A practitioner's approach*. New York, NY: McGraw-Hill Education.

Characteristics of Software

- Need to be developed or engineered, not manufactured
- Intangible
- Labour intensive
- Difficult to make changes
- Does not wear out
- Can be obsolete

Software Engineering (SE)

A process of *solving customers' problems* by the *systematic development and evolution* of *large, high-quality software systems* within *cost, time and other constraints*.

- Timothy C. Lethbridge



Graphic was produced by AI

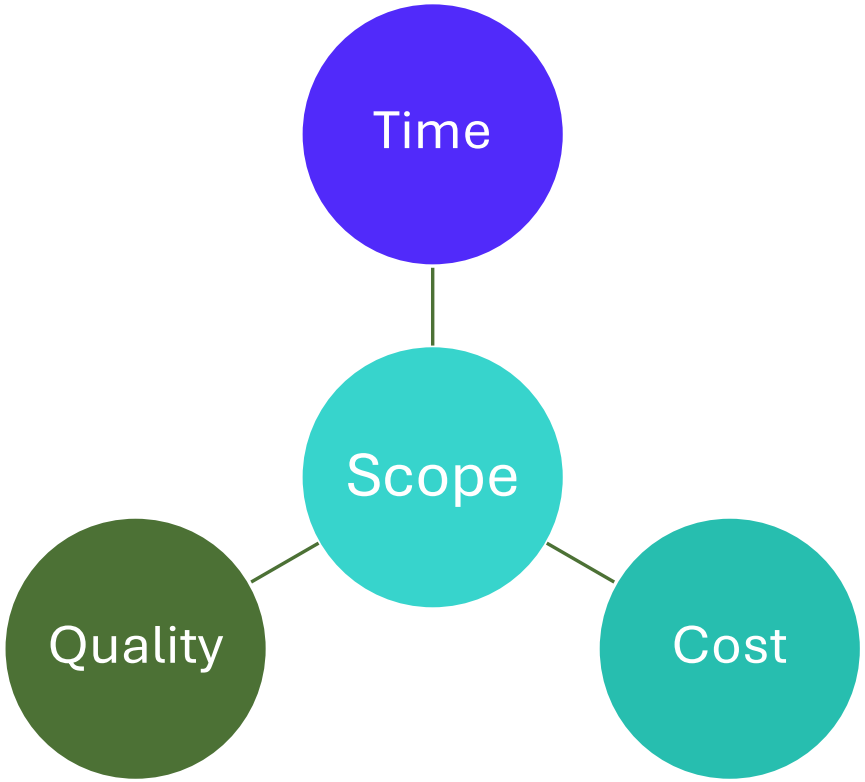
Aspects in Software Engineering

To *develop and support* a software system, these aspects are considered:

1. Technical and business processes
2. Specific methodologies and techniques
3. Product characterization and metrics for measurements
4. People skills and teamwork
5. Project coordination and management

Reference: Essentials of Software Engineering, Tsui and Karam

Software Projects



A software project needs:

- Clear objectives and goals
- Clear requirements
- Stakeholder involvement
- Executive management support
- Realistic planning
- Good management

Project Management Framework

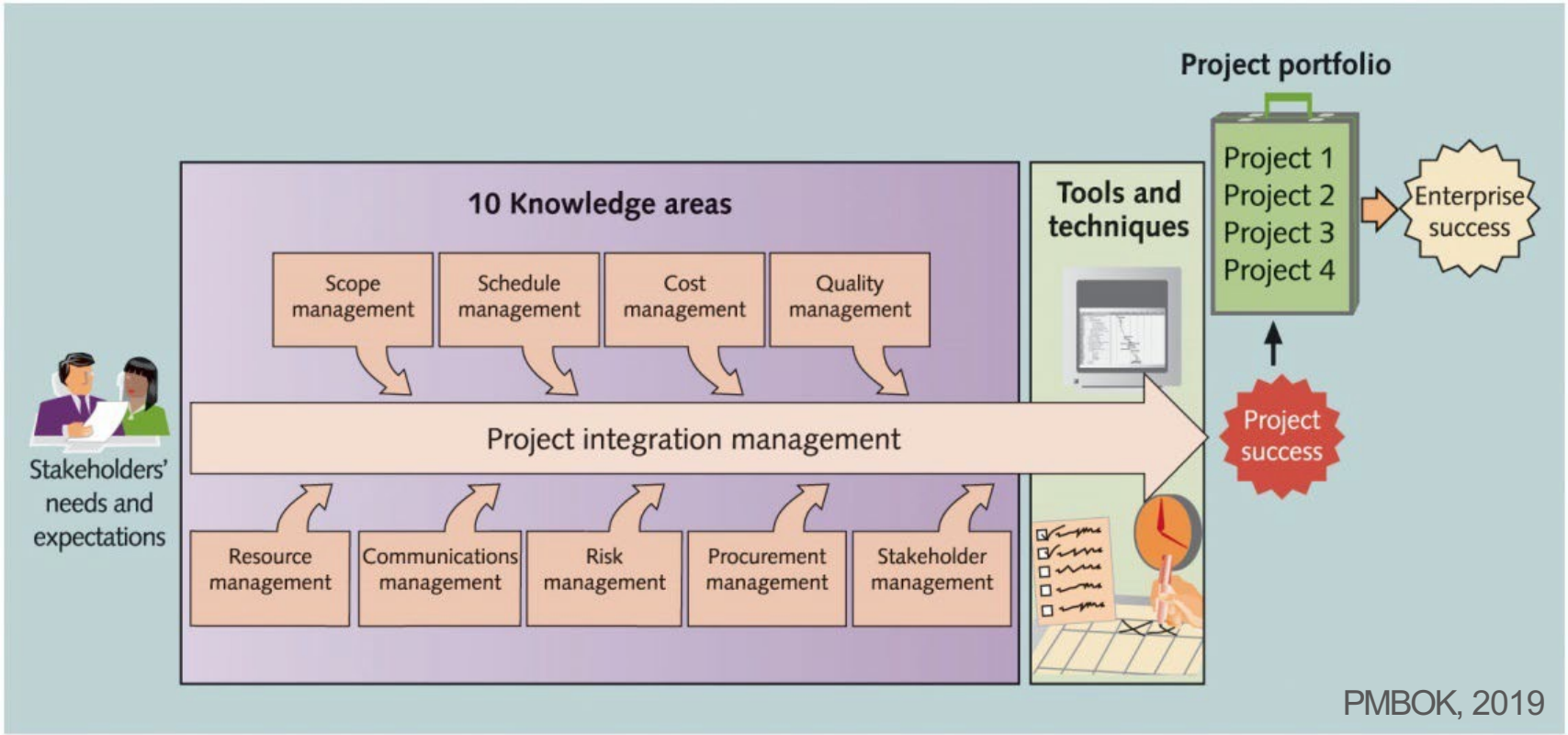


FIGURE 1-2 Project management framework

Planning (**P**OMA)

Understand the requirements of the project

Define and establish measurable goals for the project

Estimate work effort

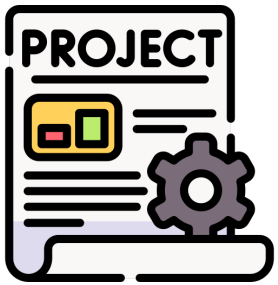
Estimate schedule

Estimate the needed resources

Determine the project resource allocations

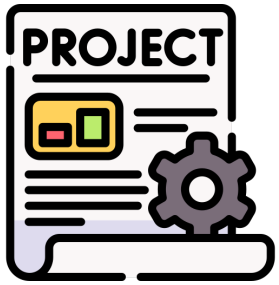
(people, process, tools, and facilities)

Identify and analyze the project risks



Project Plan

Organizing (P**O**MA)



Project Plan

Design organization structure

Hire human resources

Complete required education and training

Initiate tracking mechanisms

- Risk tracking and mitigation
- Project goal monitoring

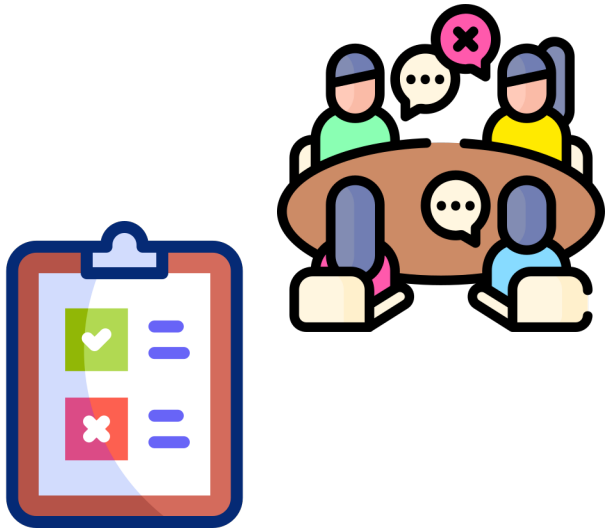
Monitoring (PO**M**A)

- Once the project is organized and set into motion, it needs to be regularly tracked to ensure that it is headed in the right direction.
- 3 main components of project monitoring:

Project status information collection

Analysis and evaluation of collected information

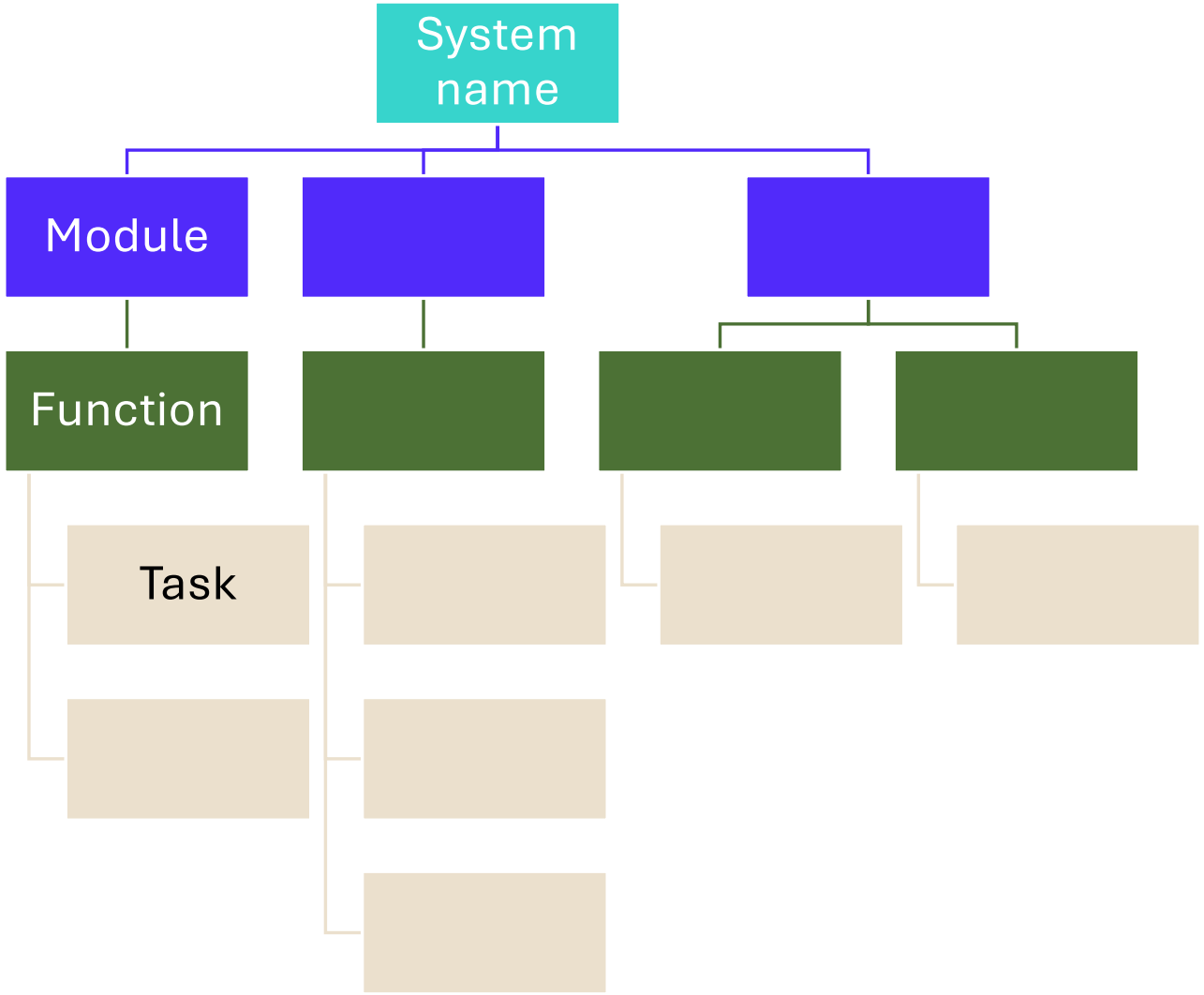
Presentation and communication of the project status



Adjusting (POM**A**)

- It is highly unlikely that a software project progresses with no problem.
- As soon as the project status suggests potential problem, we must not be afraid to make changes.
- 3 main areas of adjustments are (or combinations of)
  - Resources
  - Schedule
  - Project content

WBS



Goals of Software Project Management

Satisfy customers' needs

Project attributes are met

Target & milestones are met

Team members work effectively

Tools & resources are made available and effectively utilized



ITDS261 Introduction to Software Engineering

Lecture 2 – Software Process Models

Photo: https://www.flickr.com/photos/ninara/38707088530

Common activities in SW development

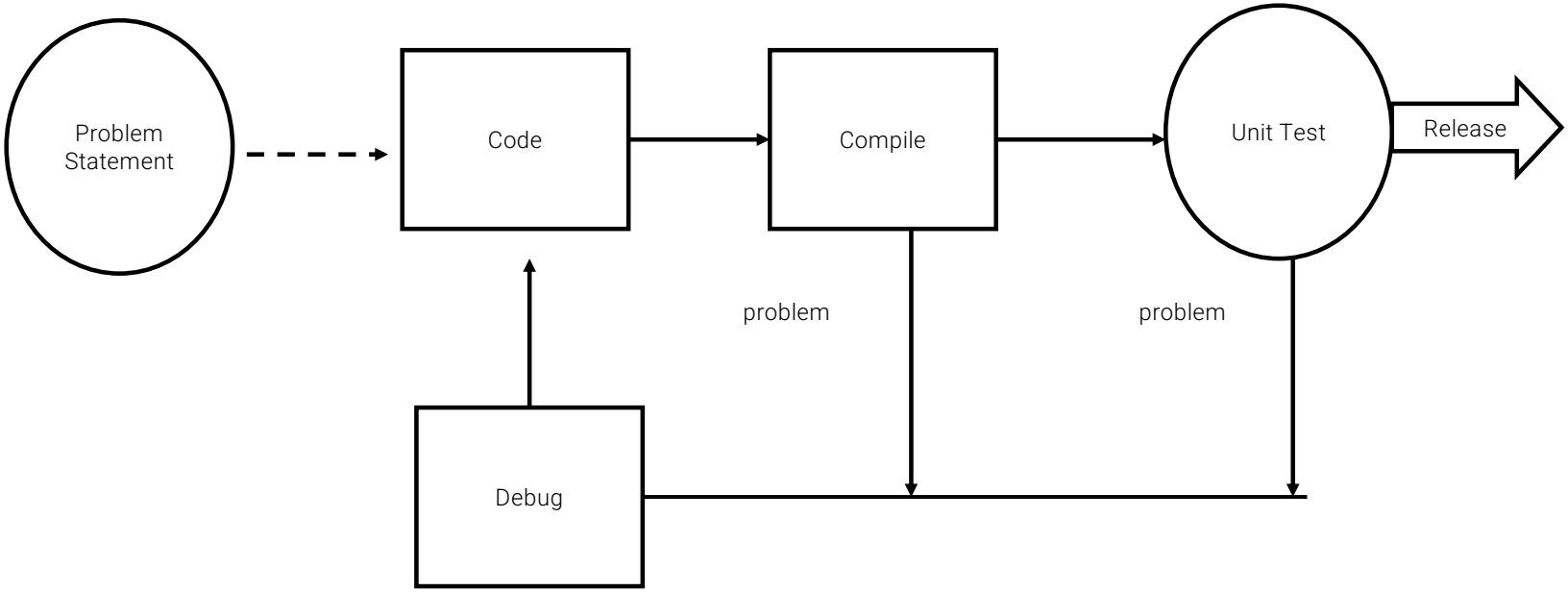
- **Requirements analysis –** Identify what the users need and define what the software should do.
- **System and software design –** Plan the structure of the system and how components will work together.
- **Implementation (coding) –** Write the program code to build the software.
- **Testing and quality assurance –** Check the software for errors and make sure it meets requirements.
- **Deployment and release –** Deliver the software for use in the real environment.
- **Maintenance and evolution –** Fix issues, update features, and adapt the software over time.

Process Models

- Process models are standardised frameworks
- Provide guidance for a systematic coordination and controlling of the tasks and of the personnel who performs the tasks
- Describe the sequence of activities, roles, artifacts, and workflows involved in the development and maintenance of a software system
  - ➢ To guide software teams in planning, executing, and managing projects
  - ➢ To ensure consistency and quality of work product
  - ➢ To facilitate communication and coordination among stakeholders

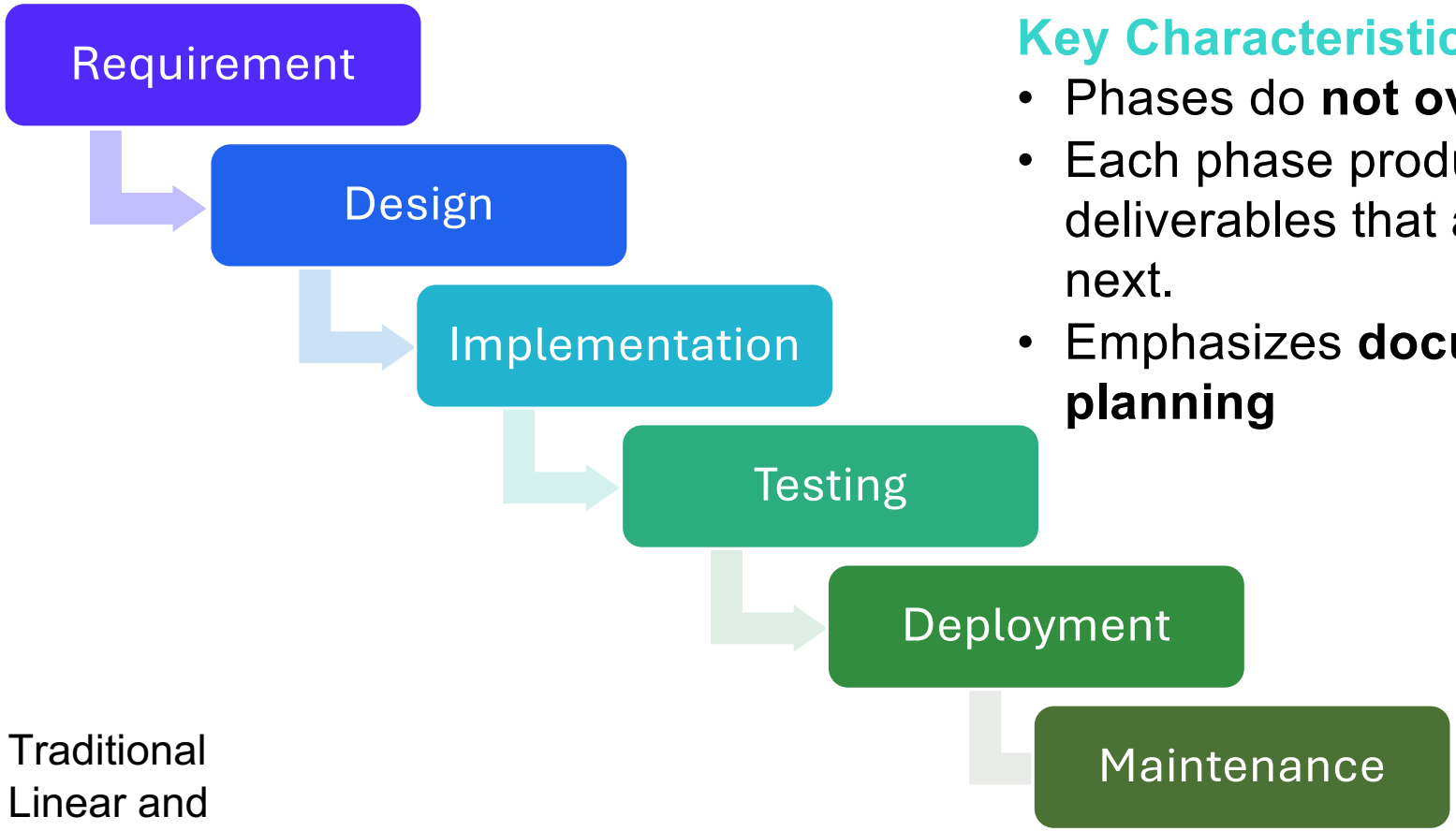
*** coordination, tasks, people ***

Code and Fix Model



1. Most people performs and follow this process, but unfortunately some *skips* unit testing or debugging
2. Also, some proceeds *without* clearly understanding the “problem statement” ---- which is requirement

Waterfall Model

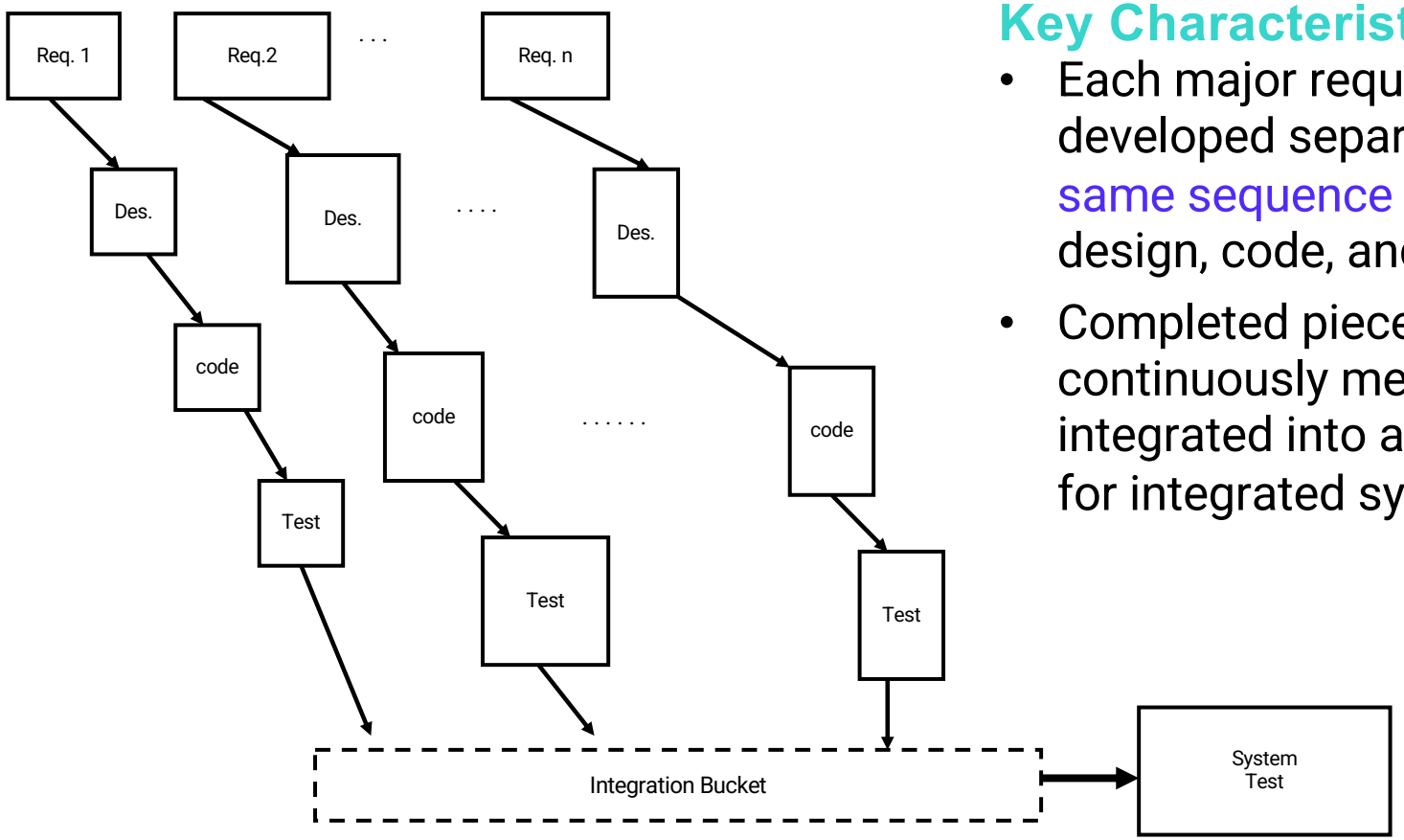


Key Characteristics:

- Phases do **not overlap**.
- Each phase produces deliverables that are inputs to the next.
- Emphasizes **documentation** and **planning**

Traditional
Linear and
sequential

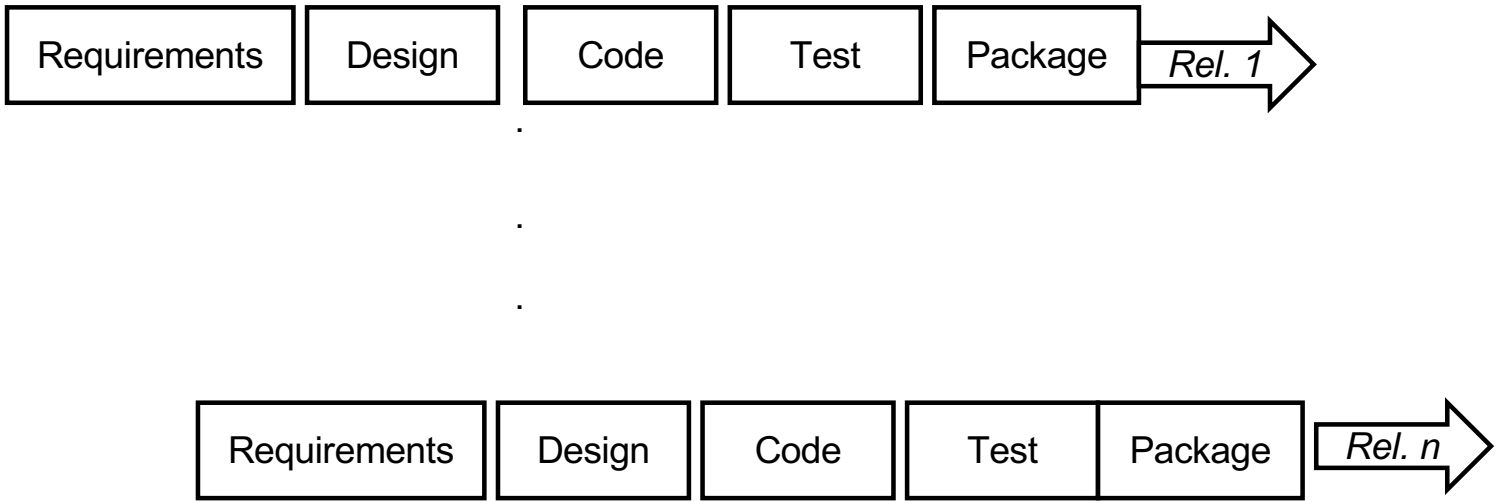
Incremental Model



Key Characteristics:

- Each major requirement/item is developed separately through the same sequence of requirement, design, code, and unit test
- Completed pieces are continuously merged and integrated into a common bucket for integrated system test

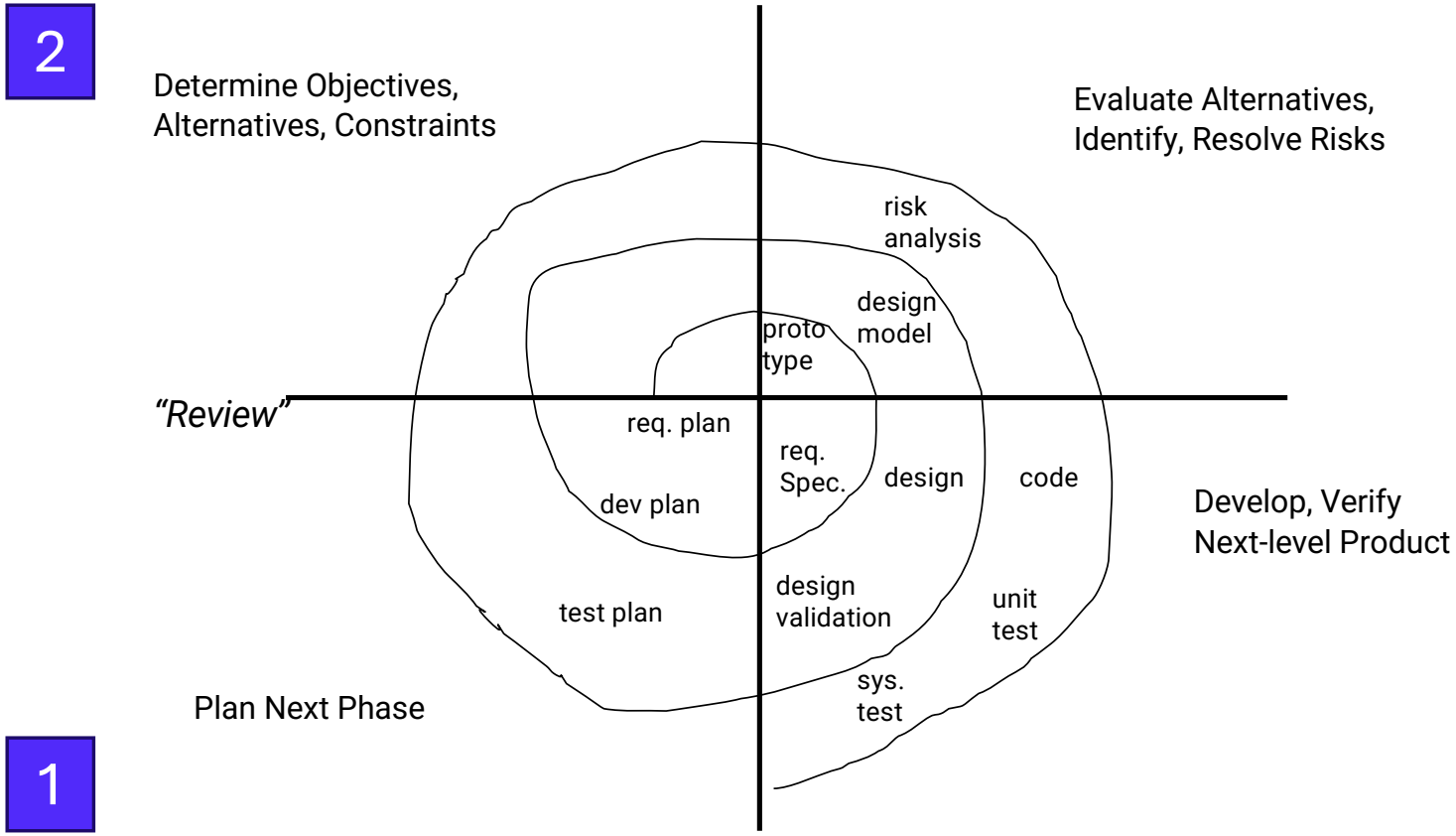
Incremental Model B - Multiple Release



Key Characteristic:

- Each small set of requirements is developed, packaged, and released in a multiple release fashion

Spiral Model

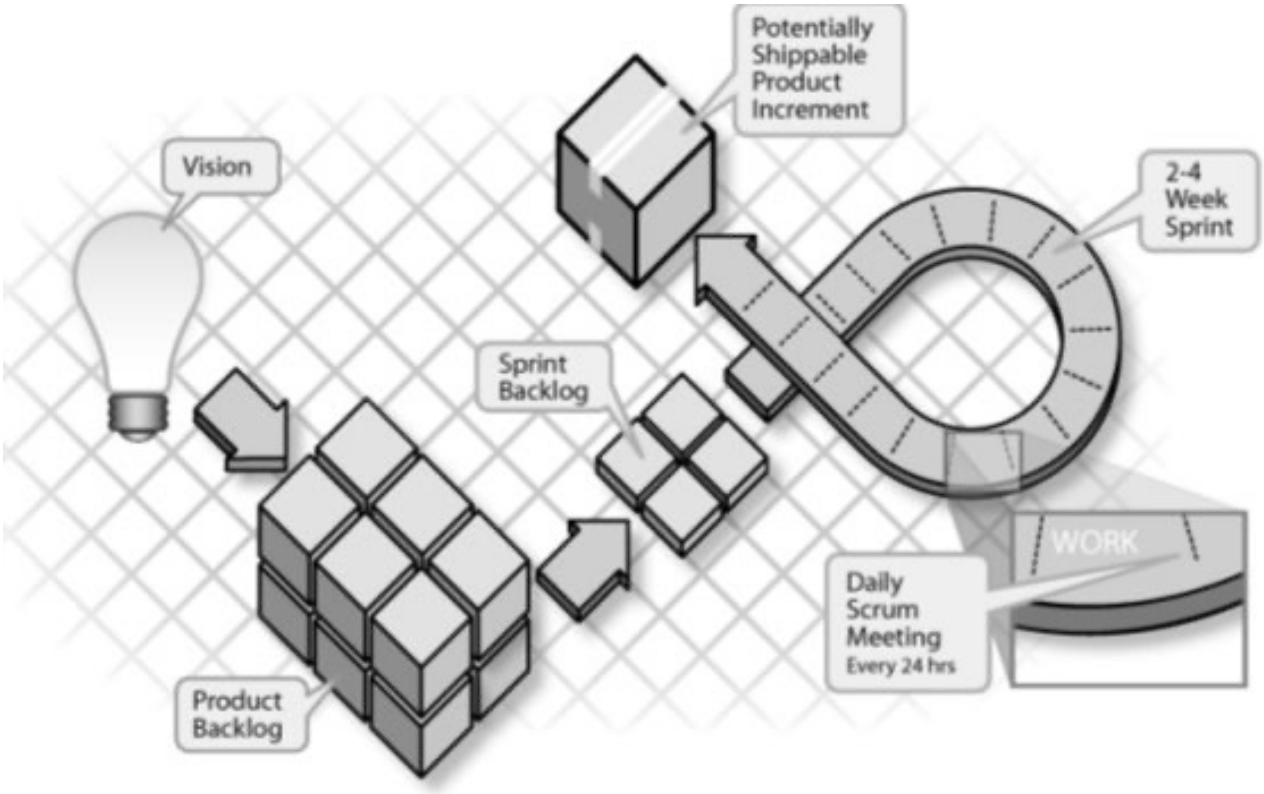


Agile

- Agile Manifesto
  - *Individuals and interactions* over processes and tools
  - *Working software* over comprehensive documentation
  - *Customer collaboration* over contract negotiation
  - *Responding to change* over following a plan

Reference: http://agilemanifesto.org

Scrum



Key Characteristics:

- Iterative development
- Incremental delivery
- Customer collaboration
- Responding to change
- Cross-functional teams
- Minimal documentation

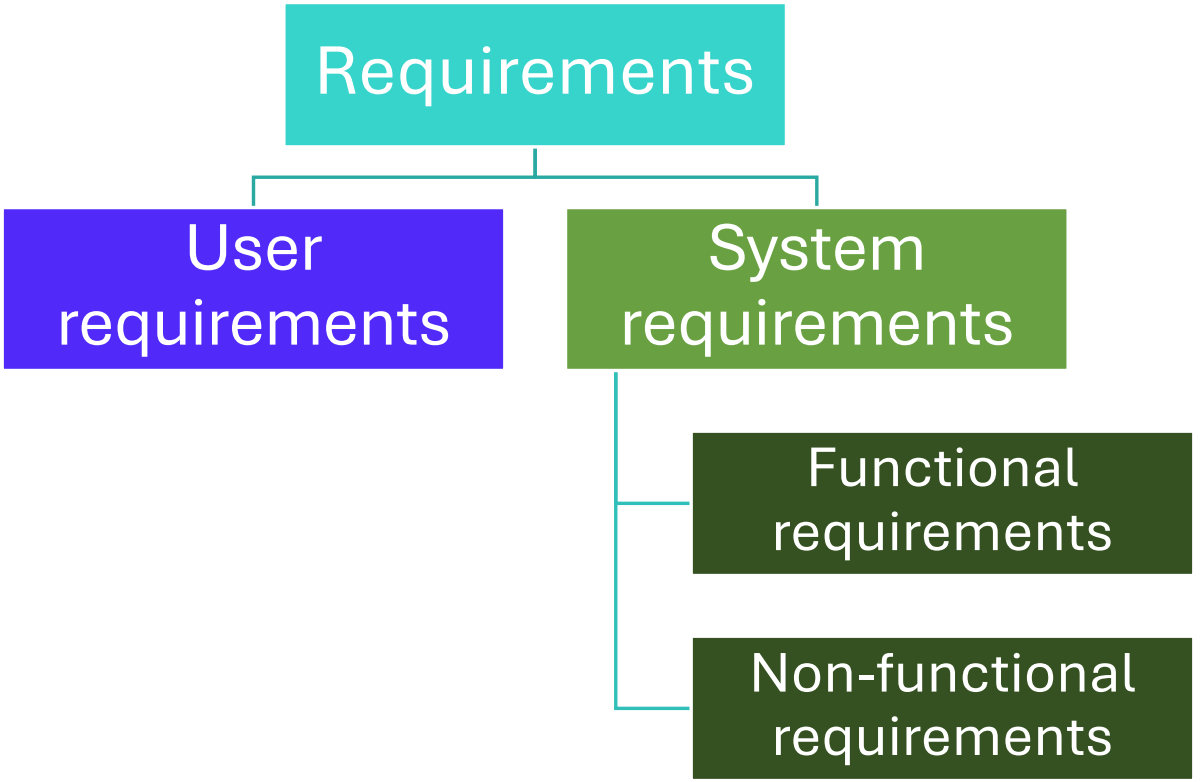
Reference: http://agilemanifesto.org; https://www.pm-partners.com.au/insights/the-agile-journey-a-scrum-overview/



ITDS261 Introduction to Software Engineering

Lecture 3 – Requirement Engineering

Types of requirements



Types of requirements

User requirements

- State what services are expected to provide to system users and the constraints under which it must operate
- May vary from broad statements of the system features required to detailed precise descriptions of the system functionality

System requirements

- State more detailed descriptions of the software system's functions, services and operational constraints
- Define exactly what is to be implemented

Functional vs Non-functional requirements

Functional requirements

- State the services that the system should provide
- Describe that the system should do
- State how the system should react to particular inputs
- State how the system should behave in particular situations

Non-functional requirements

- State the constraints on the services or functions offered by the system (e.g., response time, compatibility or memory use)
- Often apply to the system as a whole rather than individual system features or services

Requirements engineering processes

- RE is an iterative process
- Output: System requirements document
- **4 processes:**
  - Requirements elicitation
  - Requirements specification
  - Requirements validation
  - Requirements change

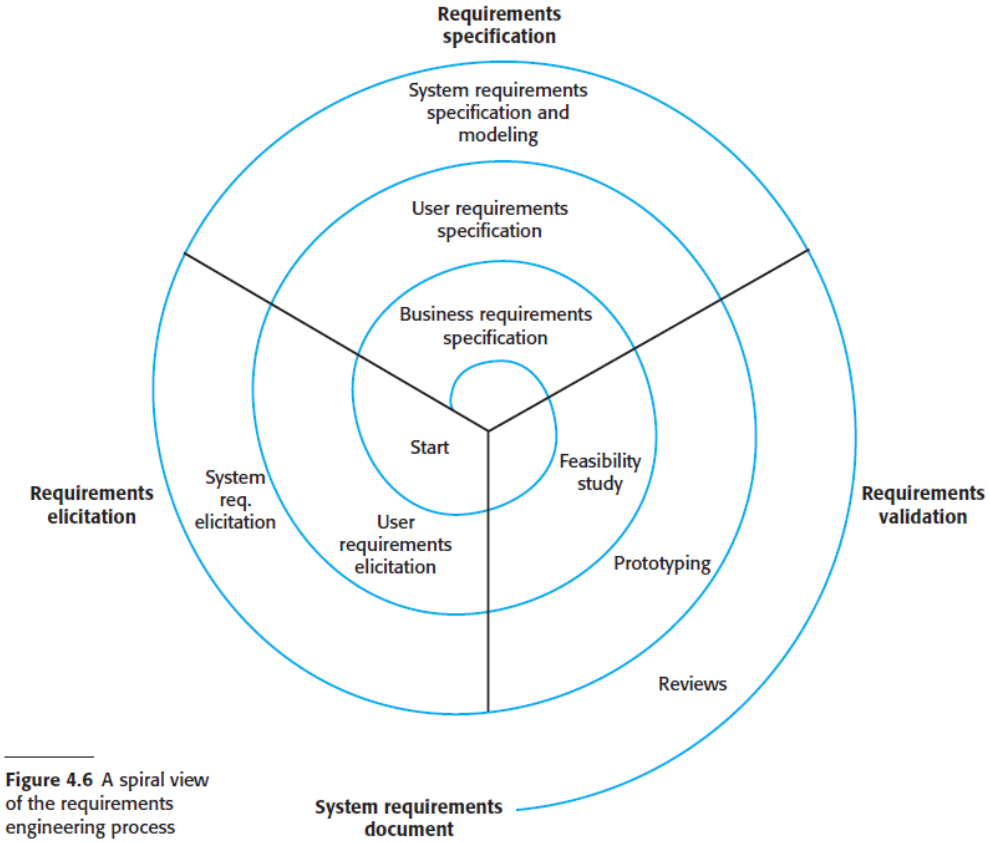
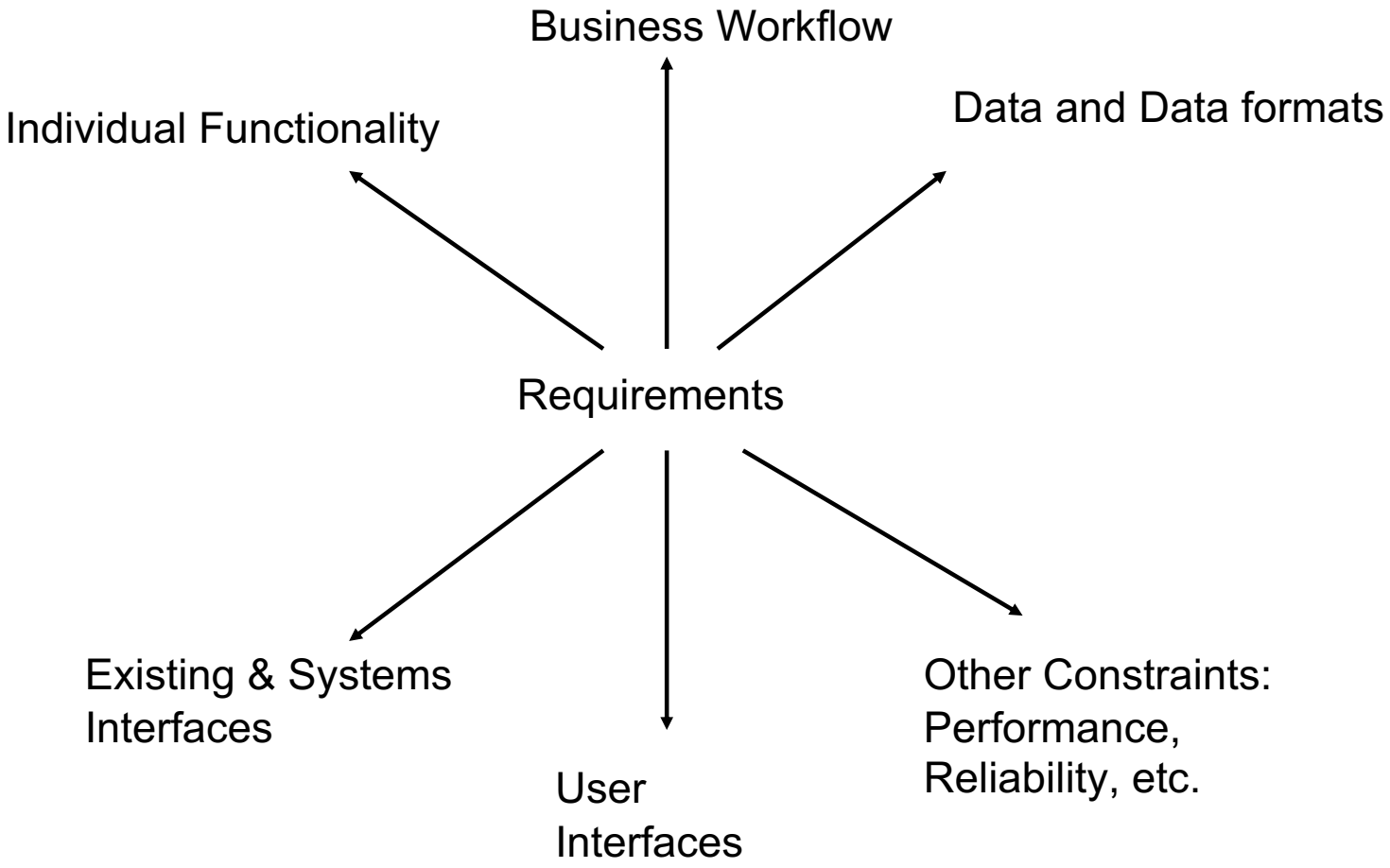


Figure 4.6 A spiral view of the requirements engineering process

6 – Dimensions of Detailed Requirements Elicitation



Requirements elicitation techniques

- Interview
- Ethnography (observation)
- Sampling of existing documentation, forms, and databases
- Research and site visits
- Questionnaires
- Joint requirements planning (JRP)

Each technique has different strengths and suitability for different scenarios

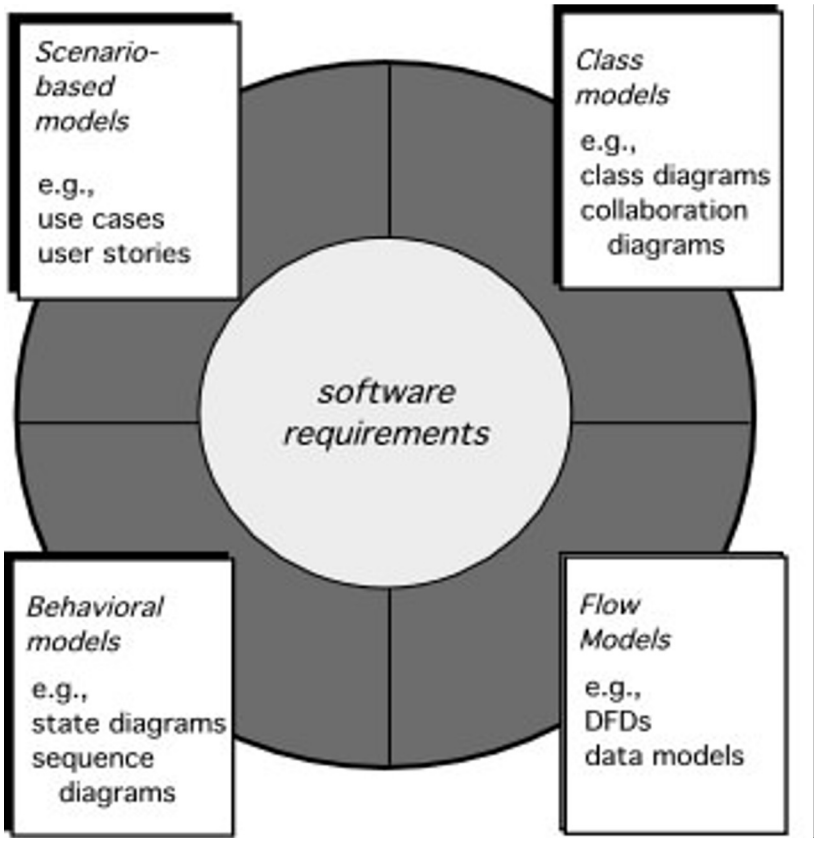
Mahidol University
Faculty of Information
and Communication Technology



ITDS261 Introduction to Software Engineering

Lecture 4 – Requirements analysis with scenario-based modeling I

Requirements Modeling Approaches



- ***Scenario-based models:*** depict requirements from the users' point of view (i.e., actors)
- ***Class models:*** represent object-oriented classes, attributes, operations and their relationships to achieve system requirements
- ***Flow models:*** represent the movement of data showing how it is transformed between processes in the system
- ***Behavioral models:*** depict how the software behaves in response to events
- ***Data models:*** depict the information domain for the problem (e.g., ER diagram)

Reference: Pressman, R.S. (2015) *Software engineering: A practitioner's approach*. New York, NY: McGraw-Hill Education.

Icon from Freepik

Scenario-based Models

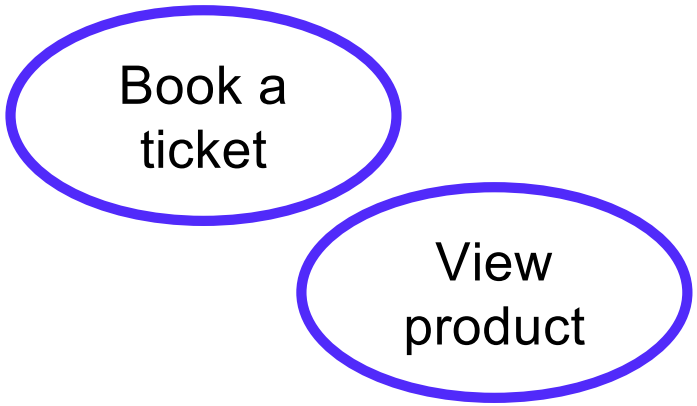
- Describe how actors will interact with the system in specific scenarios
- Focus on functional requirements from actors' point of view
- Useful for validating requirements with stakeholders
- Common technique -> Use case modelling

Basic Use Case Symbols

Use case

Use an ellipse

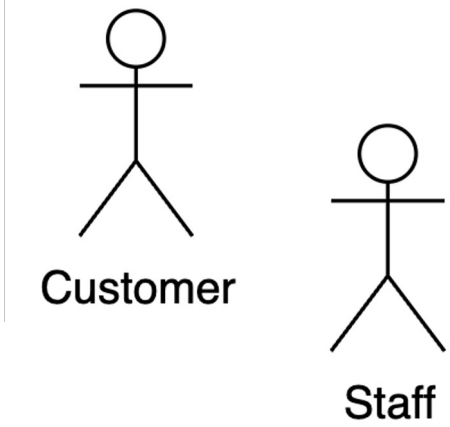
Begin with “verb”



Actor

Use a stickman symbol

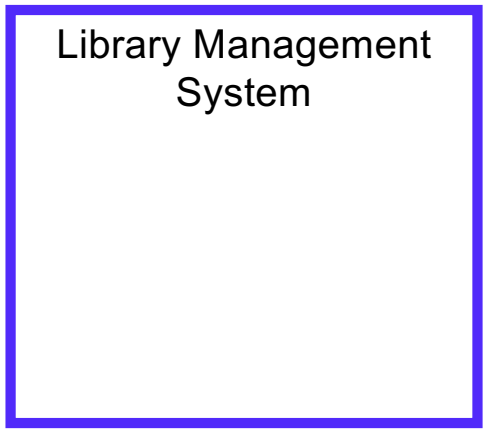
Label with “actor”



System boundary

Use a box

Label with “system name”



Use Case Relationships

- High-level relationship
  - **Association** – represents an interaction between an actor and a use case
- Detailed relationships
  - Between actors
    - **Inheritance** – a relationship where common behavior of multiple actors is moved to a shared abstract actor to avoid redundancy
  - Between use cases
    - **Includes** – a relationship that simplifies large use case by extracting common behaviours
    - **Extends** – a relationship that provides an optional behaviour from the base use case.

Key Components of a Use Case Narrative

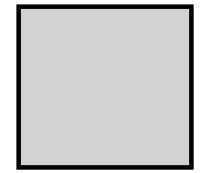
Use Case Name:	A clear, descriptive title. The name must be consistent with the one in a use case diagram.
Primary Actor:	A user or system that interacts with the use case.
Goal in Context:	The objective or purpose of the use case.
Precondition:	Conditions that must be met before the use case can begin.
Trigger:	The event that initiates the use case.
Scenario:	The typical sequence of interactions between the actor and the system. Sometimes it is called “main flow” or “basic flow”
Exception:	What happens if something goes wrong, such as an error or invalid input.
Postcondition:	The state of the system after the use case is completed.



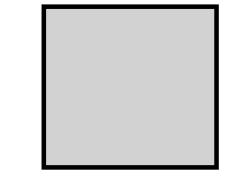
ITDS261 Introduction to Software Engineering

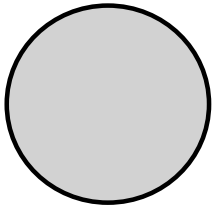
Lecture 7 – Requirements modeling with flow modeling I

Flow Modeling Notation

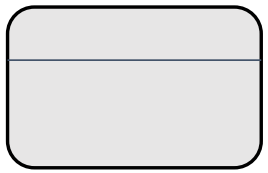


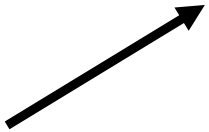
external entity



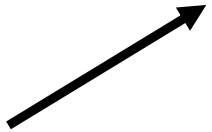


process





data flow





data store



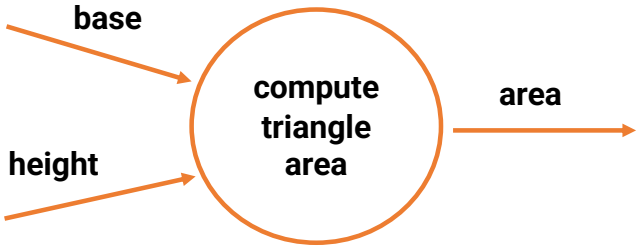
Yourdon/DeMarco's

Gane/Sarson's

Data Flow



Data flows through a system, beginning as input and transformed into output



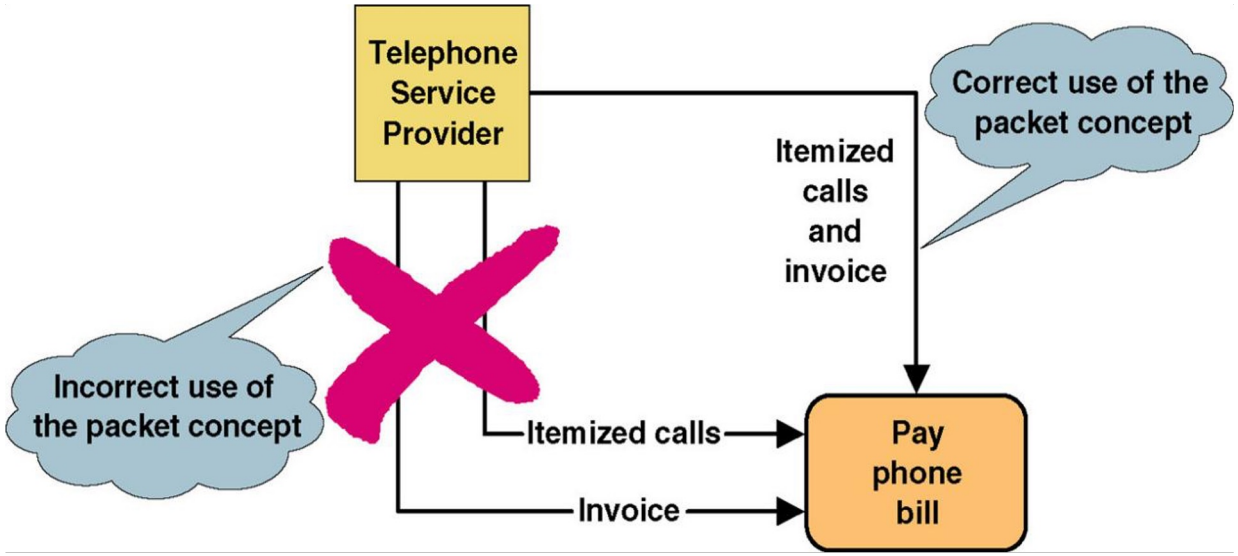
A data flow is *data in motion*.

An **input of data to a process**, or **the output of data** (or information) **from a process.**

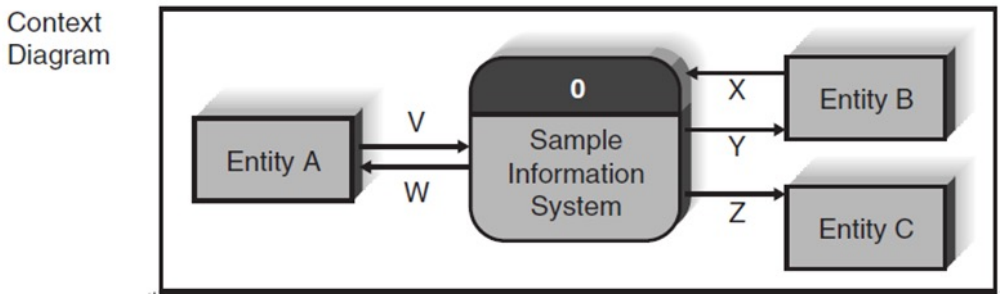
The **creation, deletion, or update of data in a file or database** (called a data store on the DFD).

Data Flow Packet Concept

- Data that should travel together should be shown as a single data flow, no matter how many physical documents might be included



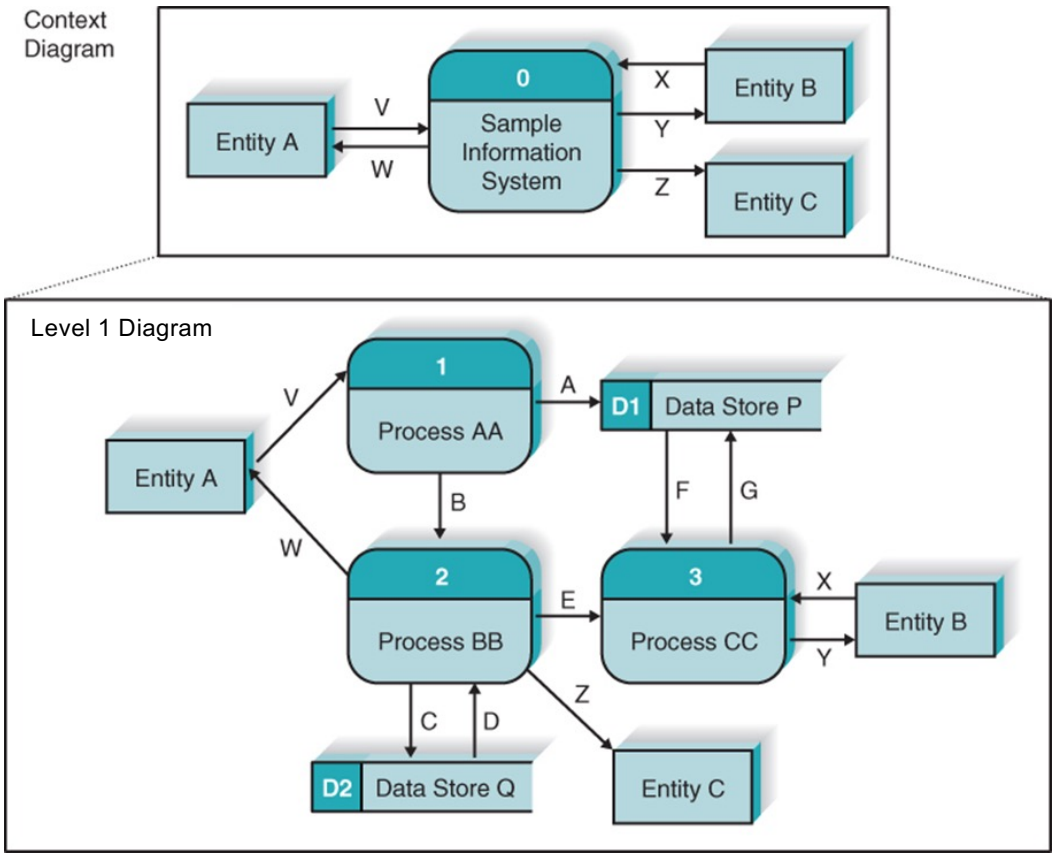
Context Diagram



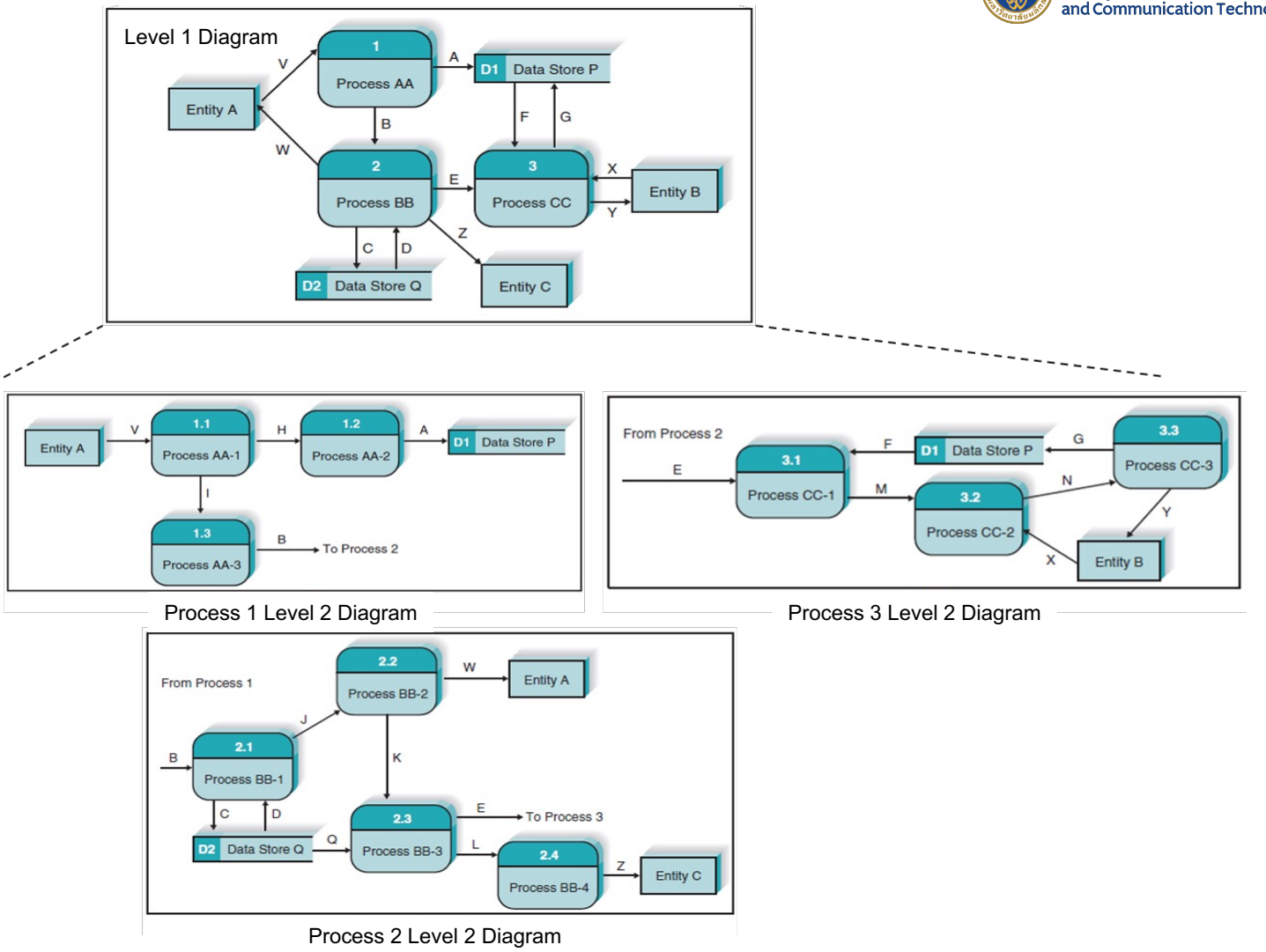
- Top-level DFD in every process model
- Shows the context into which the business process fits
- Shows the overall business process as just one process (process ‘zero’)
- Shows all the external entities that receive information from or contribute information to the system

Copyright ©2019 John Wiley & Sons, Inc.

The Data Flow Hierarchy



DFD Numbering + Hierarchical Consistency



Process 1 Level 2 Diagram

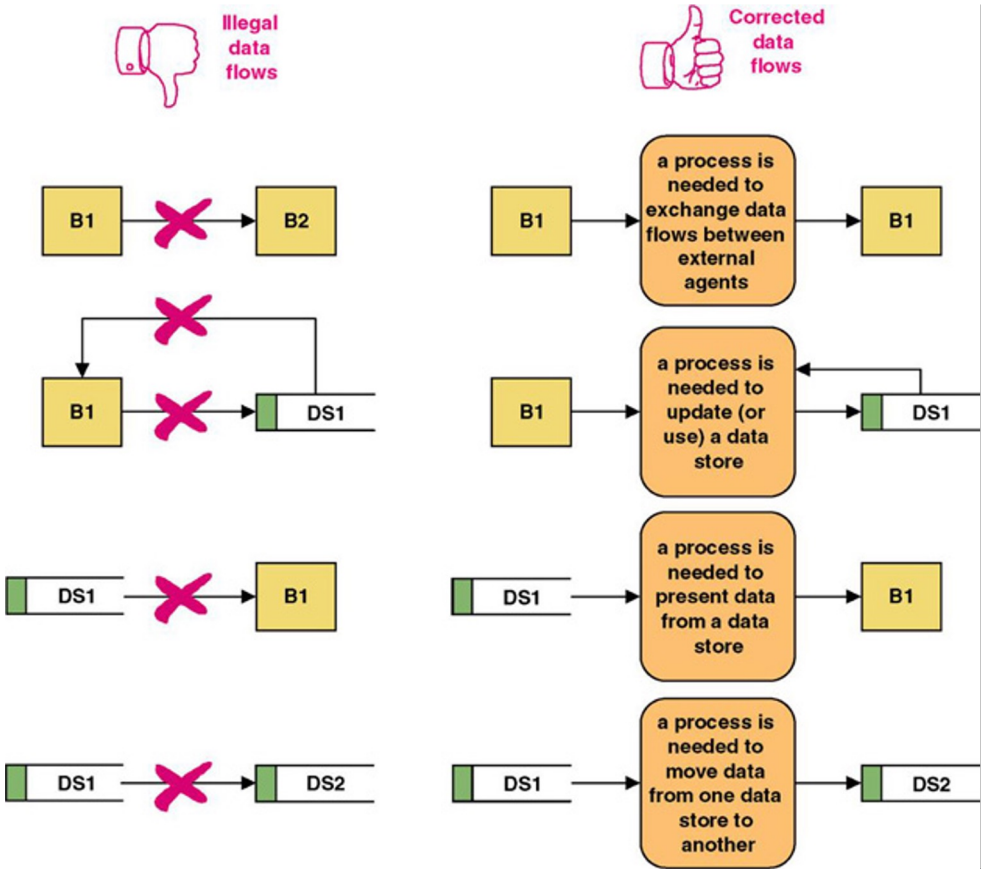
Process 3 Level 2 Diagram

Process 2 Level 2 Diagram

Copyright ©2019 John Wiley & Sons, Inc.

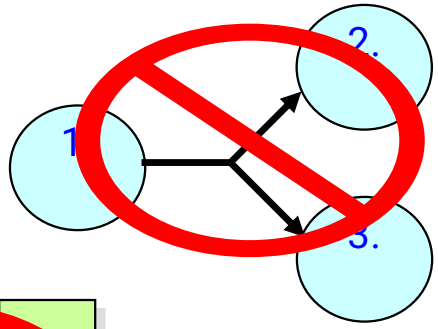
Rules for Data Flows

- A data flow should never go unnamed
- In logical modeling, data flow names should describe the data flow without describing the implementation
- All data flows must begin and/or end at a process

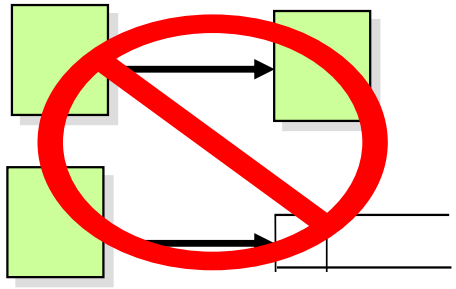


45

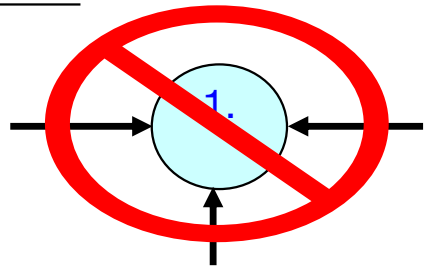
Don’t Break DFD Rules!



Data flows should not split into two or more different data flows



All data flows must EITHER originate or terminate at a process



Processes need to have at least one input data flow and one output data flow